

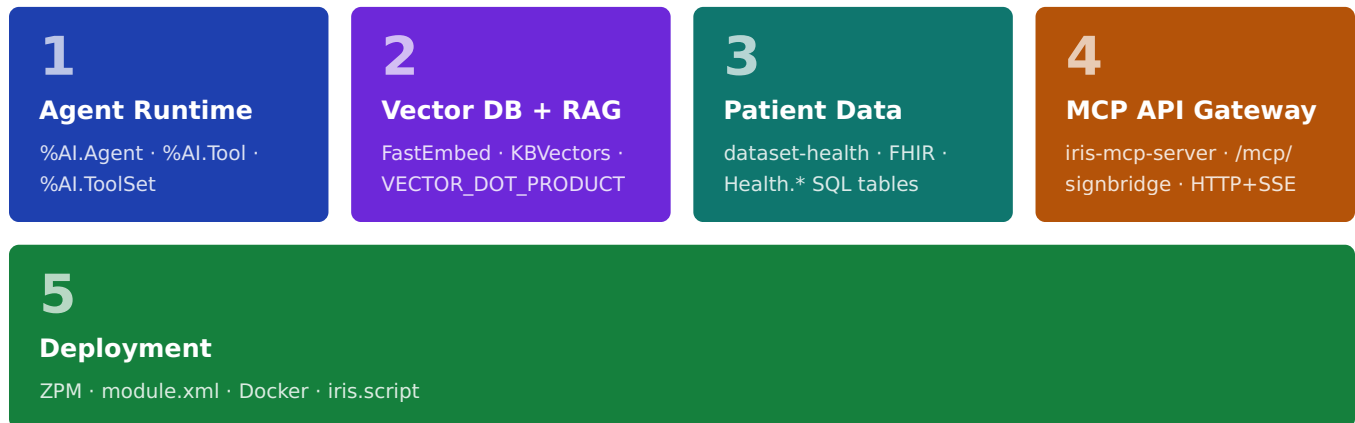
SignBridge

How We Use InterSystems IRIS

One platform. Five capabilities. Zero extra services.

The Core Argument

InterSystems IRIS is not just the database in SignBridge — it is the entire backend. What would normally require 4-5 separate services (an LLM orchestration framework, a vector database, a relational patient-data store, an API gateway, and a deployment system) is replaced by a single IRIS-for-Health 2026.2 container. The only external dependency is one OpenAI API key for LLM inference. Everything else — agent loop, RAG, SQL, HTTP transport, packaging — runs inside IRIS.



1 Agent Runtime — IRIS AI Hub SDK

What it is

IRIS 2026.2 ships a native multi-agent SDK built into the platform. Three ObjectScript superclasses form the entire framework: `%AI.Agent` runs the LLM call loop, `%AI.Tool` declares a `ClassMethod` as an LLM-callable tool, and `%AI.ToolSet` groups tools into a named set. No Python, no LangChain, no LlamaIndex required.

How SignBridge uses it

Every one of SignBridge's six agents is an ObjectScript class that extends `SignBridge.AgentBase`, which itself extends `%AI.Agent`. `AgentBase` centralises the only runtime detail that all six agents share: reading the `OPENAI_API_KEY` environment variable and constructing the provider via `%AI.Provider.Create("openai", settings)`. Each sub-agent then only needs to declare its instructions in an `XData INSTRUCTIONS` block — IRIS handles the rest.

```
// AgentBase.cls – shared base for all 6 agents (22 lines total)
Class SignBridge.AgentBase Extends %AI.Agent
{
    Parameter PROVIDER = "openai";
    Parameter MODEL     = "gpt-5-nano";

    Method %CreateProvider() As %Status {
        Set apiKey = $System.Util.GetEnviron("OPENAI_API_KEY")
        Set ..Provider = ##class(%AI.Provider).Create(..#PROVIDER, {"api_key": (apiKey)})
        Quit $$$OK
    }
}

// A specialist sub-agent – 5 lines of ObjectScript
Class SignBridge.SubAgents.TriageScorer Extends SignBridge.AgentBase
{
    XData INSTRUCTIONS { ... } // ESI 1-5 rubric written in plain English
}
```

The **Orchestrator** is the only agent that carries a toolset. Its parameter `T00LSETS = "SignBridge.ToolSet"` gives it access to 11 tools: 4 raw tools (KbSearch, SqlSelect, SqlDescribe, SqlListTables), 4 delegate tools that spawn sub-agents (AnalyzeSymptoms, SummarizeHistory, ScoreTriage, BuildGlossary), 1 report assembly tool, and 1 MCP entry point. The orchestrator's `XData INSTRUCTIONS` is a 100-line Markdown document that spells out the 9-step plan in plain English — IRIS converts it into the LLM system prompt automatically.

What IRIS does automatically

- Reads `XData INSTRUCTIONS` and sends it as the system prompt on every `Chat()` call
- Introspects each tool `ClassMethod` to auto-generate JSON schema (parameter names, types, defaults) — no hand-written schema
- Parses `tool_use` blocks from the LLM response and dispatches to the matching `ClassMethod`
- Serialises the `ClassMethod` return value back into the conversation as a `tool_result`
- Loops until the LLM stops calling tools, then returns the final text
- Tracks token counts and tool-call counts on the session object (`session.GetStats()`)

The delegate tool pattern — how sub-agents are spawned

Sub-agents do not inherit the Orchestrator's toolset. Instead, the Orchestrator calls a delegate tool (e.g., `AnalyzeSymptoms`) which uses `$CLASSMETHOD` to instantiate the sub-agent, calls `%Init()` and `CreateSession()`, sends the task JSON via `agent.Chat(session, taskJson)`, and returns the parsed response. Every sub-agent invocation is therefore a fresh agent lifecycle: created, used once, and garbage-collected.

```
// DelegateTools.cls – generic sub-agent runner (one pattern, used 4 times)
ClassMethod RunSubAgent(className As %String, taskJson As %String) As %DynamicObject {
    Set agent = $CLASSMETHOD(className, "%New")
    Set sc = agent.%Init() // reads OPENAI_API_KEY
    Set session = agent.CreateSession()
    Set response = agent.Chat(session, taskJson) // full LLM loop inside IRIS
    // parse JSON from response.Content and return it
}

// AnalyzeSymptoms delegate – assembles payload, calls runner
ClassMethod AnalyzeSymptoms(intake As %String, evidence As %String) As %DynamicObject {
    Set payload = {}
    Do payload.%Set("intake", ##class(SignBridge.DelegateTools).ParseJsonOrRaw(intake))
    Do payload.%Set("evidence", ##class(SignBridge.DelegateTools).ParseJsonOrRaw(evidence))
    Return ##class(SignBridge.DelegateTools).RunSubAgent(
        "SignBridge.SubAgents.SymptomAnalyzer", payload.%ToJson()
    )
}
```

2 Vector Database + RAG — FastEmbed inside IRIS

What it is

IRIS AI Hub ships `%AI.RAG.Embedding.FastEmbed`, a local embedding model (AllMiniLML6V2, 384 dimensions) that runs in-process inside IRIS with no external API call. Vectors are stored in a native IRIS SQL table and searched using the built-in `VECTOR_DOT_PRODUCT()` function. No Pinecone, no Chroma, no Weaviate — just IRIS SQL.

How SignBridge builds the knowledge base

`SignBridge.KB.Build()` runs once at container start (called from `iris.script`). It creates a vector store table named `SignBridge.KBVectors` with 384 dimensions, three promoted metadata columns (`category`, `condition`, `red_flag`), and then ingests 8 curated Markdown documents from `src/SignBridge/docs/`. For each document, it calls `kb.AddDocument(content, meta)` — IRIS chunks the text, runs FastEmbed to embed each chunk, and stores the vectors.

```
// KB.cls – knowledge base builder (runs once at container start)
Set emb = ##class(%AI.RAG.Embedding.FastEmbed).Create()

Set vs = ##class(%AI.RAG.VectorStore.IRIS).%New()
Set vs.TableName = "SignBridge.KBVectors"
Set vs.Dimensions = 384
Set vs.ModelName = "AllMiniLML6V2"
// Promoted metadata columns – queryable as regular SQL columns
Set fields = [
    {"name": "category", "type": "varchar(64)"},
    {"name": "condition", "type": "varchar(64)"},
    {"name": "red_flag", "type": "boolean"} // enables red-flag-only filter
]
$$$ThrowOnError(vs.Build(fields))

// Ingest one document with metadata
Set meta = {"source": "01-chest-pain-triage.md",
    "category": "cardiology", "condition": "chest_pain", "red_flag": true}
Set chunks = kb.AddDocument(content, meta) // IRIS chunks + embeds + stores
```

How SignBridge queries the knowledge base at runtime

`SignBridge.Tools.KbSearch()` is called by the Orchestrator at three points in the flow: initial symptom evidence retrieval (step 1), red-flag-only re-query when $ESI \leq 2$ (step 7), and glossary evidence retrieval (step 8). The tool first tries the high-level `kb.Search()` API; if that returns nothing it falls back to a raw `VECTOR_DOT_PRODUCT()` SQL query — both paths are IRIS-native.

```
// Tools.cls – fallback raw SQL search using VECTOR_DOT_PRODUCT
SELECT TOP 4 content, source, category, condition, red_flag,
    VECTOR_DOT_PRODUCT(embedding, TO_VECTOR(?, 'DOUBLE', 384)) AS score
FROM SignBridge.KBVectors
WHERE red_flag = 1 -- only set for the agentic re-query loop (ESI 1-2)
ORDER BY score DESC
```

The 8-document corpus

File	Category	red_flag	Used by
01-chest-pain-triage.md	cardiology	true	SymptomAnalyzer re-query loop
02-diabetes-complications.md	endocrinology	false	SymptomAnalyzer
03-hypertension.md	cardiology	false	SymptomAnalyzer
04-asthma-respiratory.md	pulmonology	false	SymptomAnalyzer
05-esi-triage-rubric.md	triage	false	TriageScorer context
06-medication-glossary.md	glossary	false	InterpreterBriefer
07-deaf-patient-communication.md	communication	false	InterpreterBriefer
08-sign-language-medical-glossary.md	glossary	false	InterpreterBriefer

3 Patient Data — FHIR SQL via dataset-health

What it is

`dataset-health` is an InterSystems ZPM package that installs a synthetic FHIR-style patient dataset directly into IRIS as standard relational tables under the `Health.*` schema. There is no separate database — the same `%SQL.Statement` interface used for everything else in IRIS queries these tables.

How SignBridge uses it

The Orchestrator follows a 3-step schema discovery pattern before querying. First it calls `SqlListTables("Health%")` to learn which tables exist in this dataset. Then it calls `SqlDescribe(tableName)` on one or two of them to understand the column names. Finally it runs 1-3 targeted `SqlSelect()` queries filtered to the patient's ID. This means the Orchestrator adapts to whatever version of `dataset-health` is installed — it never hard-codes column names.

```
// Tools.cls – SqlListTables: schema discovery (step 3 in the orchestrator plan)
SELECT TABLE_SCHEMA, TABLE_NAME
FROM INFORMATION_SCHEMA.TABLES
WHERE TABLE_SCHEMA LIKE ?      -- "Health%"
ORDER BY TABLE_SCHEMA, TABLE_NAME

// SqlDescribe: column discovery (so the LLM knows what to SELECT)
SELECT COLUMN_NAME, DATA_TYPE, IS_NULLABLE
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = ? AND TABLE_NAME = ?
ORDER BY ORDINAL_POSITION

// SqlSelect: actual patient query – guarded against writes
// The guard strips the SQL, uppercases it, and rejects any non-SELECT statement
// before %SQL.Statement ever sees it.
```

The read-only SQL guard

Because the LLM constructs SQL strings, `SqlSelect()` enforces a hard read-only rule in ObjectScript before any query reaches `%SQL.Statement`. It uppercases and strips whitespace from the SQL, verifies it starts with `SELECT`, rejects multi-statement input (anything after a `;`), and scans for a blocklist of write keywords: `INSERT`, `UPDATE`, `DELETE`, `DROP`, `ALTER`, `CREATE`, `TRUNCATE`, `GRANT`, `REVOKE`, `MERGE`, `CALL`, `EXEC`. Any match returns an error immediately — the LLM cannot write data.

4 MCP API Gateway — iris-mcp-server + IRIS Web Layer

What it is

IRIS hosts the HTTP endpoint that Streamlit connects to. Two components work together: **iris-mcp-server**, a Rust binary that ships inside the IRIS container and bridges HTTP+SSE to IRIS's native WgProto protocol; and the IRIS web layer, which hosts the `/mcp/signbridge` CSP web application that routes calls into our `SignBridge.MCPService` class.

How the endpoint is registered

`SignBridge.MCPSetup.Run()` is invoked by ZPM at install time. It switches to the `%SYS` namespace and calls `Security.Applications.Create("/mcp/signbridge", props)` with four key properties: `DispatchClass = "SignBridge.MCPService"`, `Type = 18` (MCP service type), `Namespace = "IRISAPP"`, and `AuthEnabled = 64` (unauthenticated for the hackathon). After this, IRIS's web server on port 52773 handles `GET /mcp/signbridge` requests automatically.

```
// MCPSetup.cls – registers the web app (invoked once by ZPM)
Set props("DispatchClass") = "SignBridge.MCPService"
Set props("Type")           = 18    // MCP service type – IRIS knows to route MCP protocol
Set props("AuthEnabled")    = 64    // unauthenticated
Set props("Namespace")     = "IRISAPP"
Set props("MatchRoles")     = ":%All"
##class(Security.Applications).Create("/mcp/signbridge", .props)

// MCPService.cls – one parameter, IRIS generates the entire tool manifest
Class SignBridge.MCPService Extends %AI.MCP.Service
{
    Parameter SPECIFICATION = "SignBridge.ToolSet";
}
```

The single tool exposed to clients

Although `SignBridge.ToolSet` declares 11 tools internally, only `ProcessIntake` (in `SignBridge.Entry`) is meaningfully called by clients. Streamlit sends one JSON intake object to `ProcessIntake` and waits. The entire 6-agent orchestration — RAG, SQL, 9+ LLM calls, the optional re-query loop — happens inside IRIS between those two events. The client never sees the sub-agents.

```
// Entry.cls – the only method Streamlit ever calls
ClassMethod ProcessIntake(intakeJson As %String) As %DynamicObject {
    Set agent = ##class(SignBridge.Orchestrator).%New()
    Set sc    = agent.%Init()
    Set session = agent.CreateSession()
    Set response = agent.Chat(session, intakeJson) // all 9 steps happen here
    Set result.brief = response.Content           // Markdown brief → Streamlit
    Set result.tokens = session.GetStats()        // token usage for the UI
    Return result
}
```

Connection path end-to-end

Streamlit (port 8501) → HTTP POST to iris-mcp-server (port 8080) → WgProto to IRIS native server (port 1972) → IRISAPP namespace → `SignBridge.MCPService` → `SignBridge.Entry.ProcessIntake()` → `SignBridge.Orchestrator.Chat()` → 9-step agent loop → Markdown brief returned back along the same path.

5 Deployment — ZPM + Docker

What it is

SignBridge ships as a standard IRIS ZPM module. `module.xml` declares all 15 ObjectScript classes as resources. ZPM compiles them, resolves dependencies, and invokes two setup methods on install — all in one command: `zpm load /app`.

What happens at container start (iris.script)

The container's init script runs five things in order, all inside IRIS:

1 ZPM installer download

IRIS fetches the latest ZPM installer from `pm.community.intersystems.com` via `%Net.HttpRequest` and loads it — no pre-installed package manager needed.

2 dataset-health install

`zpm "install dataset-health"` pulls the FHIR patient dataset from the InterSystems package registry and creates all `Health.*` tables in the IRISAPP namespace.

3 zpm load — compile + register

`zpm "load /home/irisowner/dev/ -v"` reads `module.xml`, compiles all 15 SignBridge classes, and invokes the two `<Invoke>` entries: `Sample.MCPSetup.Run()` and `SignBridge.MCPSetup.Run()` — creating both MCP web applications.

4 SignBridge.KB.Build()

Calls the knowledge base builder directly: `do ##class(SignBridge.KB).Build()`. This creates the `SignBridge.KBVectors` table, runs FastEmbed to embed all 8 corpus documents, and stores the vectors. After this line the RAG system is live.

5 iris-mcp-server start

The Rust bridge is started separately via Docker so it persists beyond the init script. It reads `config.toml`, connects to IRIS on port 1972, and begins accepting HTTP on port 8080.

```
// module.xml — declares all 15 classes and two setup invocations
<Module>
  <SourcesRoot>src</SourcesRoot>
  <Resource Name="SignBridge.AgentBase.CLS"/>
  <Resource Name="SignBridge.Orchestrator.CLS"/>
  <Resource Name="SignBridge.SubAgents.SymptomAnalyzer.CLS"/>
  <Resource Name="SignBridge.SubAgents.HistoryAgent.CLS"/>
  <Resource Name="SignBridge.SubAgents.TriageScorer.CLS"/>
  <Resource Name="SignBridge.SubAgents.InterpreterBriefer.CLS"/>
  <Resource Name="SignBridge.SubAgents.ReportWriterAgent.CLS"/>
  <Resource Name="SignBridge.Tools.CLS"/>
  <Resource Name="SignBridge.DelegateTools.CLS"/>
  <Resource Name="SignBridge.ReportWriter.CLS"/>
  <Resource Name="SignBridge.Entry.CLS"/>
  <Resource Name="SignBridge.ToolSet.CLS"/>
  <Resource Name="SignBridge.KB.CLS"/>
  <Resource Name="SignBridge.MCPService.CLS"/>
  <Resource Name="SignBridge.MCPSetup.CLS"/>
  <Invoke Method="Run" Class="SignBridge.MCPSetup"/>
</Module>
```

The Five-in-One Advantage

Without IRIS you'd need	With IRIS
LangChain / LlamaIndex for agent orchestration + tool dispatch	<code>%AI.Agent</code> + <code>%AI.Tool</code> — native ObjectScript, no Python dependency
Pinecone / Chroma / Weaviate for vector storage + similarity search	<code>SignBridge.KBVectors</code> <code>table</code> + <code>VECTOR_DOT_PRODUCT()</code> — same SQL interface
PostgreSQL / MySQL for patient records	IRIS SQL — same engine, same <code>%SQL.Statement</code> API as everywhere else
FastAPI / Express + MCP SDK for the HTTP tool endpoint	<code>%AI.MCP.Service</code> + <code>iris-mcp-server</code> — one Parameter, IRIS generates the manifest
Helm / Terraform / Ansible for deployment	<code>module.xml</code> + <code>zpm load</code> + <code>docker compose up</code> — one command deploys everything

The result: SignBridge is roughly 1,200 lines of ObjectScript across 15 classes with zero Python service code on the backend. The only Python in the repository is the Streamlit front-end and two test scripts. Everything that matters runs inside IRIS.